claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_416cc0e5-a35\agent-a70ca07c93bb74bf7.jsonl`
- SHA-256: `c58e8b6c7384100ad8ed6bdba0b1907df91b4056fe3854734e83dfb23aa20ddf`
- Source modified: `2026-06-22T09:48:51+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:47:06.910Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

{
 "title": "LOW: Cost billing uses kernel uptime, which could differ slightly from actual wall-clock time",
 "file": "scripts/nightly_reelcount.py",
 "line": "73-90",
 "severity": "low",
 "detail": "The cost_report function accepts billed_seconds from the orchestrator (run_on_vm.sh line 80, which calculates UPTIME = $(date +%s) - BOOT_EPOCH). This is kernel time, not wall-clock. On a heavily loaded system or if the system clock jumps (NTP adjustment), the calculated UPTIME could be inaccurate by a few seconds. For a 5-minute run, this is negligible (<1% error), but for a longer run it could accumulate. The manifeset allows specifying machine_usd_per_hr, but there's no documented way to audit or reconcile the actual GCP billing vs. the reported UPTIME.",
 "fix": "Document in nightly_reelcount.py that reported cost is a best-effort estimate based on VM uptime, not the authoritative GCP bill. Add a comment: 'Compare this cost estimate against the actual GCP billing statement (Cost Management API) for drift.' Optionally: in register_scheduler.sh, add a post-run step that queries GCP's Cost Management API to report the actual billed amount."
}

2. assistant (2026-06-22T09:47:08.663Z)

I'll review this finding by examining the actual code to verify whether this is a real defect.

3. user (2026-06-22T09:47:10.050Z)

```
1       """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM nightly).
2
3       Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
```

```
deployment serves
4       it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
5       vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6       trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7       needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8       owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10      What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11        ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12          Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13          **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see §guard).
14        ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15          ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16        ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18      Extensible pattern (add a video = one manifest row + re-freeze):
19        ``eval_baselines/reelcount_manifest.json``  ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20        ``eval_baselines/reelcount_baseline.json``  ? the frozen expected counts (regenerated,
never hand-edited)
21
22      Usage
23      -----
24      Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25          python scripts/nightly_reelcount.py --update-baseline
26
27      Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28          python scripts/nightly_reelcount.py --email
29
30      Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31      (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32      """
33      from __future__ import annotations
34
35      import argparse
36      import datetime as _dt
37      import json
38      import os
39      import sys
40      from dataclasses import dataclass, field
41      from typing import Any, Dict, List, Optional, Tuple
42
43      REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44      if REPO_ROOT not in sys.path:
45          sys.path.insert(0, REPO_ROOT)
46
47      # billing is pure (typing only) ? safe to import at module top.
48      from backend import billing  # noqa: E402
49
```

```
    DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
    DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
    DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")

    #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
    QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
    DEFAULT_QUALITY_TOL = 0.02     # quality is float/labeled ? a looser tripwire than the
exact reel-count
    DEFAULT_FX_INR_PER_USD = 83.0
    #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
    DEFAULT_MACHINE_USD_PER_HR = 0.35


    @dataclass(frozen=True)
    class Tolerances:
        quality_tol: float = DEFAULT_QUALITY_TOL

        def to_dict(self) -> Dict[str, Any]:
            return {"quality_tol": self.quality_tol}


    # ---------------------------------------------------------------------------- #
    # Pure cost math (wraps backend.billing ? no IO).
    # ---------------------------------------------------------------------------- #
    def cost_report(billed_seconds: float, machine_usd_per_hr: float,
                    fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
        """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).

        ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
        otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
        rate_per_sec = float(machine_usd_per_hr) / 3600.0
        usage = {billing.WALL_SECONDS: float(billed_seconds)}
        rates = {billing.WALL_SECONDS: rate_per_sec}
        usd, breakdown = billing.compute_cost_usd(usage, rates)
        return {
            "billed_seconds": round(float(billed_seconds), 1),
            "gpu_seconds": round(float(gpu_seconds), 1),
            "machine_usd_per_hr": float(machine_usd_per_hr),
            "usd": usd,
            "inr": billing.to_inr(usd, fx_inr_per_usd),
            "breakdown": breakdown,
        }


    # ---------------------------------------------------------------------------- #
    # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
    # ---------------------------------------------------------------------------- #
    def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
                          cost: Dict[str, Any]) -> Dict[str, Any]:
        """Compact, comparable snapshot from per-video run results.

        Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
        A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
never hides it)."""
        per_video: Dict[str, Any] = {}
        for r in run_results:
            per_video[r["name"]] = {
                "reel_count": r.get("reel_count"),
                "ok": bool(r.get("ok", False)),
```

```
                "error": r.get("error"),
                "quality": r.get("quality"),
            }
        return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
                "n": len(per_video)}


    @dataclass
    class Finding:
        video: str
        metric: str                # "reel_count" | "error" | a quality key | "segmenter" |
"corpus"
        kind: str                  # "regression" | "improvement" | "stale"
        baseline: Optional[float] = None
        current: Optional[float] = None
        detail: str = ""

        def message(self) -> str:
            if self.kind == "stale":
                return f"[STALE] {self.metric}: {self.detail}"
            if self.metric == "error":
                return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
            b = "?" if self.baseline is None else f"{self.baseline:g}"
            c = "?" if self.current is None else f"{self.current:g}"
            tag = "REGRESSION" if self.kind == "regression" else "improvement"
            return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"


    @dataclass
    class NightlyResult:
        findings: List[Finding] = field(default_factory=list)
        added: List[str] = field(default_factory=list)
        removed: List[str] = field(default_factory=list)

        @property
        def regressions(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "regression"]

        @property
        def improvements(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "improvement"]

        @property
        def stale(self) -> List[Finding]:
            return [f for f in self.findings if f.kind == "stale"]

        def overall_status(self) -> str:
            if self.regressions:
                return "REGRESSION"
            if self.stale:
                return "STALE"
            return "PASS"

        def exit_code(self) -> int:
            if self.regressions:
                return 1
            if self.stale:
                return 4
            return 0


    def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
        """Diff current run vs frozen baseline.

```

```
        * Segmenter changed ? STALE (numbers not comparable; re-freeze).
        * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
        * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
          quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
        """
        res = NightlyResult()
        if baseline.get("segmenter") != current.get("segmenter"):
            res.findings.append(Finding("", "segmenter", "stale",
                                        detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}"))
            return res

        b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
        res.added = sorted(set(c_pv) - set(b_pv))
        res.removed = sorted(set(b_pv) - set(c_pv))
        if res.added or res.removed:
            res.findings.append(Finding("", "corpus", "stale",
                                        detail=f"added={res.added} removed={res.removed}"))

        for v in sorted(set(b_pv) & set(c_pv)):
            b, c = b_pv[v], c_pv[v]
            # 1) pipeline must still succeed
            if not c.get("ok", False) or c.get("reel_count") is None:
                res.findings.append(Finding(v, "error", "regression",
                                            detail=str(c.get("error") or "no reel_count
produced")))
                continue
            # 2) reel count ? EXACT
            bc, cc = b.get("reel_count"), c.get("reel_count")
            if bc is not None and cc is not None and cc != bc:
                res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
            # 3) quality metrics ? tolerance (only if both sides have them)
            bq, cq = b.get("quality") or {}, c.get("quality") or {}
            for m in QUALITY_HIGHER:
                bv, cv = bq.get(m), cq.get(m)
                if bv is None or cv is None:
                    continue
                if cv < bv - tols.quality_tol:
                    res.findings.append(Finding(v, m, "regression", bv, cv))
                elif cv > bv + tols.quality_tol:
                    res.findings.append(Finding(v, m, "improvement", bv, cv))
        return res


    # ------------------------------------------------------------------------ #
    # Report formatting (pure).
    # ------------------------------------------------------------------------ #
    def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
        if not qd or qd.get(key) is None:
            return "?"
        return f"{qd[key]:.3f}"


    def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
                      result: NightlyResult, date: str, head: str) -> str:
        status = result.overall_status()
        badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
                 "STALE": "? STALE (re-freeze the baseline)"}[status]
        cost = current.get("cost", {})
        L: List[str] = [
            f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
            "",
```

```
            f"**Status: {badge}**  ·  exit code {result.exit_code()}  ·  segmenter
`{current.get('segmenter')}`",
            "",
            f"- HEAD `{head}`  ·  baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",
            f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})**  ·  "
            f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
            "",
        ]
        if result.regressions:
            L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
+ [""]
        if result.stale:
            L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
            L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]

        # Per-video table.
        b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
        L += ["## Per-video", "",
              "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
              "|---|---|---|---|---|"]
        for v in sorted(set(b_pv) | set(c_pv)):
            b, c = b_pv.get(v, {}), c_pv.get(v, {})
            bc = b.get("reel_count")
            cc = c.get("reel_count")
            rc = f"{'?' if bc is None else bc}?{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
            bq, cq = b.get("quality"), c.get("quality")
            tf = f"{_q(bq, 'tol_f1')}?{_q(cq, 'tol_f1')}"
            f5 = f"{_q(bq, 'f1@0.5')}?{_q(cq, 'f1@0.5')}"
            vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else "?"
            L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
        L.append("")

        if result.improvements:
            L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
                + [f"- {f.message()}" for f in result.improvements] + [""]
        L += ["---",
              "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
              "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
        return "\n".join(L)


    # ------------------------------------------------------------------------ #
    # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
    # ------------------------------------------------------------------------ #
    def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
        """Load ground-truth rally intervals from a golden-features JSON (``gts`` key) or a
rallies CSV
        (``start,end`` columns / pairs). Returns None when no labels are configured."""
        if not labels_ref:
            return None
        from backend.storage import fetch
        local = fetch(labels_ref)
        if local.endswith(".json"):
            with open(local, encoding="utf-8") as fh:
                data = json.load(fh)
            return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
        # CSV: header start,end (or first two numeric columns)
```

```
        import csv
        out: List[Tuple[float, float]] = []
        with open(local, encoding="utf-8") as fh:
            for row in csv.reader(fh):
                try:
                    out.append((float(row[0]), float(row[1])))
                except (ValueError, IndexError):
                    continue
        return out


    def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
        """``(start, end)`` pairs from segmenter result dicts, tolerant of the two key
conventions in the
        codebase (``start_time``/``end_time`` ? motion/DB shape ? or ``start``/``end``).
Used only for the
        optional quality metrics; the reel COUNT is ``len(segs)`` regardless, so a key
mismatch degrades
        quality scoring gracefully (skipped) without ever miscounting reels."""
        out: List[Tuple[float, float]] = []
        for s in segs:
            start = s.get("start_time", s.get("start"))
            end = s.get("end_time", s.get("end"))
            if start is not None and end is not None:
                out.append((float(start), float(end)))
        return out


    def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
                      rerun_on_mismatch: bool = True,
                      baseline_count: Optional[int] = None) -> Dict[str, Any]:
        """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.

        The re-run-once guard (the one known non-determinism ? a transient
``add_play_segment``?None drop,
        database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't
        reproduce, a real change will."""
        import random
        import time

        from backend.eval import rally_seg_eval
        from backend.infrastructure.database import Database
        from backend.pipeline.segmenters import segmenter_registry
        from backend.results import Failure
        from backend.storage import fetch
        from backend.utils.hashing import compute_video_id

        name = video["name"]
        try:
            local = fetch(video["uri"])
        except Exception as e:  # noqa: BLE001 ? surface, never hide
            return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
{e}",
                    "quality": None, "wall_seconds": 0.0}

        gts = _load_gts(video.get("labels_uri"))
        seg_cls = segmenter_registry.get(segmenter)
        if not seg_cls:
            return {"name": name, "reel_count": None, "ok": False,
                    "error": f"segmenter '{segmenter}' not registered", "quality": None,
"wall_seconds": 0.0}

        def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
```

```
float]]], float, Optional[str]]:
            random.seed(0)  # close motion.py's unseeded metadata RNG so secondary fields
are reproducible too
            db = Database(db_path)
            video_id = compute_video_id(local)
            t0 = time.monotonic()
            result = seg_cls(db, config).process_video(video_id, local)
            wall = time.monotonic() - t0
            if isinstance(result, Failure):
                return None, None, wall, getattr(result, "message", "segmenter failed")
            segs = result.value if hasattr(result, "value") else result
            intervals = _segments_to_intervals(segs)
            return len(segs), intervals, wall, None

        import tempfile
        total_wall = 0.0
        with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
            count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
            total_wall += wall
            if err is not None:
                return {"name": name, "reel_count": None, "ok": False, "error": err,
                        "quality": None, "wall_seconds": round(total_wall, 2)}
            # re-run-once guard for the transient DB-drop flake
            if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
                count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
                total_wall += wall2
                if err2 is None and count2 is not None:
                    count, intervals = count2, intervals2  # the reproduced (or corrected)
value

            quality = None
            if gts and intervals is not None:
                row = rally_seg_eval.score_intervals(intervals, gts)
                quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
"tol_precision", "tol_recall")
                           if k in row}

        return {"name": name, "reel_count": count, "ok": True, "error": None,
                "quality": quality, "wall_seconds": round(total_wall, 2)}


    def load_manifest(path: str) -> Dict[str, Any]:
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
        """Run every manifest video through the configured pipeline. Returns (run_results,
total_wall_s)."""
        from backend.config import load_config

        config = load_config()
        # apply manifest config overrides onto the typed config (dotted keys)
        for dotted, val in (manifest.get("config_overrides") or {}).items():
            _set_dotted(config, dotted, val)
        segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
"default_segmenter", "motion")
        b_pv = (baseline or {}).get("per_video", {})
        results: List[Dict[str, Any]] = []
        total_wall = 0.0
        for v in manifest.get("videos", []):
            base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
            r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
            total_wall += float(r.get("wall_seconds") or 0.0)
```

```
            results.append(r)
        return results, total_wall


    def _set_dotted(config: Any, dotted: str, value: Any) -> None:
        """Best-effort dotted-path set onto a pydantic config (e.g.
'indexing.motion_threshold')."""
        parts = dotted.split(".")
        obj = config
        for p in parts[:-1]:
            obj = getattr(obj, p, None)
            if obj is None:
                return
        try:
            setattr(obj, parts[-1], value)
        except Exception:  # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
crash the run
            pass


    def _git_head() -> str:
        import subprocess
        try:
            out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
                                 capture_output=True, text=True, timeout=10)
            return out.stdout.strip() or "?"
        except Exception:  # noqa: BLE001
            return "?"


    def load_baseline(path: str) -> Optional[Dict[str, Any]]:
        if not os.path.exists(path):
            return None
        with open(path, encoding="utf-8") as fh:
            return json.load(fh)


    def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:
        out = dict(snapshot)
        out["generated_at"] = _dt.date.today().isoformat()
        out["git_commit"] = _git_head()
        out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python
scripts/nightly_reelcount.py "
                       "--update-baseline` after an INTENDED engine change or when adding a
video. Tied to the "
                       "configured+checkpointed engine + the manifest corpus.")
        out.pop("cost", None)  # cost is per-run, not part of the frozen baseline
        os.makedirs(os.path.dirname(path), exist_ok=True)
        tmp = path + ".tmp"
        with open(tmp, "w", encoding="utf-8") as fh:
            json.dump(out, fh, indent=2, sort_keys=True)
            fh.write("\n")
        os.replace(tmp, path)


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
quality + cost.")
        p.add_argument("--manifest", default=DEFAULT_MANIFEST)
        p.add_argument("--baseline", default=DEFAULT_BASELINE)
        p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
        p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
        p.add_argument("--vm-uptime-seconds", type=float, default=None,
                       help="authoritative VM uptime (boot?delete) for the cost line; "
                            "defaults to the in-process pipeline wall time (a lower bound)")
```

```
        p.add_argument("--no-rerun-guard", action="store_true",
                       help="disable the re-run-once-on-mismatch guard (the DB-drop flake
mitigation)")
        p.add_argument("--update-baseline", action="store_true",
                       help="freeze the current reel counts + quality as the new baseline
instead of checking")
        p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
        p.add_argument("--email-to", default=None, help="override the report recipient")
        p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
        p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
        return p


    def main(argv: Optional[List[str]] = None) -> int:
        import logging
        logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
        log = logging.getLogger("nightly_reelcount")
        args = build_parser().parse_args(argv)
        tols = Tolerances(quality_tol=args.quality_tol)
        date = args.date or _dt.date.today().isoformat()

        if not os.path.exists(args.manifest):
            log.error("manifest not found: %s", args.manifest)
            return 2
        manifest = load_manifest(args.manifest)
        baseline = load_baseline(args.baseline)

        try:
            run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
        except Exception as e:  # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
            log.error("run failed: %s", e)
            return 2
        if not run_results:
            log.error("no videos in manifest %s", args.manifest)
            return 2

        billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
        cost = cost_report(
            billed_seconds=billed,
            machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
            fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
            gpu_seconds=total_wall,
        )
        segmenter = manifest.get("segmenter", "motion")
        snapshot = summarize_results(run_results, segmenter, cost)
        snapshot["git_commit"] = _git_head()
        snapshot["generated_at"] = date

        if args.update_baseline:
            save_baseline(args.baseline, snapshot)
            log.info("baseline frozen ? %s", args.baseline)
            for r in run_results:
                log.info("  %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
            return 0

        if baseline is None:
            log.error("no baseline at %s ? freeze one with --update-baseline (on the
```

```
configured engine).", args.baseline)
515             return 3
516
517         result = compare(baseline, snapshot, tols)
518         report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])
519
520         if not args.no_report:
521             os.makedirs(args.report_dir, exist_ok=True)
522             md_path = os.path.join(args.report_dir, f"{date}.md")
523             with open(md_path, "w", encoding="utf-8") as fh:
524                 fh.write(report + "\n")
525             with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526                 json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                            "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                           fh, indent=2, sort_keys=True)
529             log.info("report ? %s", md_path)
530
531         print(report)
532
533         if args.email:
534             try:
535                 from scripts.nightly_email import send_report
536                 subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537                 sent = send_report(subject, report, to=args.email_to)
538                 log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539             except Exception as e:  # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540                 log.error("email failed (report still written): %s", e)
541
542         code = result.exit_code()
543         if code == 1:
544             log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)
545         elif code == 4:
546             log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547         else:
548             log.info("nightly reel-count check PASSED  (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549         return code
550
551
552     if __name__ == "__main__":
553         raise SystemExit(main())
554
```

4. user (2026-06-22T09:47:10.311Z)

```
1       #!/usr/bin/env bash
2       # Startup-script for the NIGHTLY engine-regression VM. Runs UNATTENDED on boot, then
DELETES the VM
3       # (cost rule: a running GPU VM is the only real cost). Survives SSH drops (it's a
startup-script, not
4       # an interactive session). Reads its config from instance metadata (set by launch.sh).
5       #
6       # Flow:  Ops Agent  ?  pull repo @ HEAD from GCS  ?  build venv + WASB env  ?  stage
weights  ?
7       #        run scripts/nightly_reelcount.py on the gs:// golden clips (emails the report)
?
8       #        upload report to GCS  ?  SELF-DELETE (even on failure, via the EXIT trap).
```

```
9       #
10      # Metadata keys (launch.sh sets these): repo-tgz-uri, git-sha, gcs-bucket, report-
prefix,
11      #   weights-uri, email (true/false). NOT verified in CI ? dry-run via launch.sh on a
real VM first.
12      set -uo pipefail
13      exec > >(tee -a /var/log/nightly-regression.log) 2>&1
14      echo "===== nightly-regression startup $(date -u +%FT%TZ) ====="
15      BOOT_EPOCH=$(date +%s)
16
17      md() { curl -s -H "Metadata-Flavor: Google"
"http://metadata.google.internal/computeMetadata/v1/$1"; }
18      NAME=$(md instance/name)
19      ZONE=$(md instance/zone | awk -F/ '{print $NF}')
20      REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
21      GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
22      BUCKET=$(md instance/attributes/gcs-bucket || true)
23      REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
24      WEIGHTS_URI=$(md instance/attributes/weights-uri || true)
25      DO_EMAIL=$(md instance/attributes/email || echo true)
26      SMTP_SECRET=$(md instance/attributes/smtp-secret || true)
27      SMTP_USER=$(md instance/attributes/smtp-user || true)
28      EMAIL_TO=$(md instance/attributes/email-to || echo avi.dullu@gmail.com)
29      DATE=$(date -u +%F)
30
31      # ALWAYS delete the VM on exit (success OR failure) ? never leave a billing GPU VM
behind.
32      cleanup() {
33        echo "===== self-delete $NAME ($ZONE) ====="
34        gcloud compute instances delete "$NAME" --zone="$ZONE" --quiet || \
35          echo "WARN: self-delete failed ? DELETE MANUALLY: gcloud compute instances delete
$NAME --zone=$ZONE -q"
36      }
37      trap cleanup EXIT
38
39      fail_email() {  # best-effort failure alert (the report email won't have fired on an
early crash)
40        echo "RUN FAILED ? attempting a failure alert"
41        if [ "$DO_EMAIL" = "true" ] && [ -d ~/rally ]; then
42          cd ~/rally 2>/dev/null && . .venv/bin/activate 2>/dev/null && \
43          NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
44          python - <<PY || true
45      from scripts.nightly_email import send_report
46      send_report("[KhelSutra nightly] ERROR ? engine regression VM run failed ($DATE)",
47                  "The nightly engine-regression run failed on the VM before producing a
report.\n"
48                  "See gs://$BUCKET/$REPORT_PREFIX/$DATE/ and the VM serial
log.\nGIT_SHA=$GIT_SHA", to="$EMAIL_TO")
49      PY
50        fi
51      }
52      trap 'fail_email' ERR
53
54      # 1) Observability (Ops Agent) ? same as create_gpu_vm.sh bakes in.
55      curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && \
56        sudo bash add-google-cloud-ops-agent-repo.sh --also-install || echo "WARN: ops-agent
install failed (non-fatal)"
57
58      # 2) Pull the repo @ HEAD (staged to GCS by launch.sh ? no GitHub auth needed; uses the
worker SA).
59      mkdir -p ~/rally && cd ~/rally
60      gcloud storage cp "$REPO_TGZ" /tmp/rally.tgz
61      tar -xzf /tmp/rally.tgz -C ~/rally
62      echo "$GIT_SHA" > ~/rally/GIT_SHA
```

```
63
64      # 3) Build the env + native WASB (provider-agnostic DO scripts, reused on GCP per
deploy/gcp/README §4).
65      sudo bash deploy/digitalocean/mount_scratch.sh 2>/dev/null || true
66      export TMPDIR=/scratch 2>/dev/null || true
67      ./deploy/digitalocean/bootstrap.sh --gpu
68      . .venv/bin/activate
69      pip install -e '.[storage-gcs]'
70      bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned
nonzero ? continuing"
71
72      # 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4).
73      if [ -n "$WEIGHTS_URI" ]; then
74        mkdir -p ~/models/WASB-SBDT/pretrained_weights
75        gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
76        export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
77      fi
78
79      # 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
uptime so far.
80      UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
81      EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
82      set +e
83      NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
84        python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
85      RUN_RC=$?
86      set -e
87      echo "runner exit code: $RUN_RC  (0 pass / 1 regression / 4 stale)"
88
89      # 6) Upload the dated report to GCS for the record (alongside the email).
90      if [ -n "$BUCKET" ] && [ -d output/nightly_reelcount ]; then
91        gcloud storage cp -r "output/nightly_reelcount/$DATE."*
"gs://$BUCKET/$REPORT_PREFIX/$DATE/" || \
92          gcloud storage cp -r output/nightly_reelcount "gs://$BUCKET/$REPORT_PREFIX/$DATE/"
|| true
93      fi
94
95      echo "===== nightly-regression done rc=$RUN_RC $(date -u +%FT%TZ) ====="
96      trap - ERR   # disable the failure-email trap; the EXIT (self-delete) trap stays
97      # the EXIT trap deletes the VM
98
```

5. user (2026-06-22T09:47:11.188Z)

```
1       {
2         "_doc": "Nightly E2E engine reel-count manifest (scripts/nightly_reelcount.py). The
configured+checkpointed pipeline runs on each video on a GCP GPU VM; the reel count + quality
metrics are asserted vs eval_baselines/reelcount_baseline.json. ADD A VIDEO = append one row +
re-freeze: `python scripts/nightly_reelcount.py --update-baseline` (commit the updated
baseline). URIs may be gs:// (recommended, co-located with the VM), gdrive://, or local paths.
labels_uri (optional) is a golden-features JSON (gts key) or a start,end CSV ? present =>
quality metrics (tol_f1/f1@0.5/f1@0.25) are scored too. EDIT the example rows below to your
actual short (~5-6 min) OWNED, minors-free clips before enabling.",
3         "segmenter": "wasb_hybrid",
4         "detector_impl": "native",
5         "config_overrides": {
6           "indexing.default_segmenter": "wasb_hybrid",
7           "indexing.detector_impl": "native"
8         },
9         "machine_usd_per_hr": 0.70,
10        "fx_inr_per_usd": 83.0,
11        "videos": [
12          {
```

```
13            "name": "GX030094",
14            "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX030094.mp4",
15            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX030094_golden_features.json"
16          },
17          {
18            "name": "Badminton_BXH_2",
19            "uri": "gs://khelsutra-rally-corpus/nightly/videos/Badminton_BXH_2.mp4",
20            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/Badminton_BXH_2_golden_features.json"
21          },
22          {
23            "name": "GX010128",
24            "uri": "gs://khelsutra-rally-corpus/nightly/videos/GX010128.mp4",
25            "labels_uri": "gs://khelsutra-rally-
corpus/nightly/labels/GX010128_golden_features.json"
26          }
27        ]
28      }
29
```

6. assistant (2026-06-22T09:47:13.405Z)

Now let me examine the specific lines cited in the finding and verify the claim:

7. user (2026-06-22T09:47:15.303Z)

```
73      def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                      fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75          """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).
76
77          ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
78          otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
79          rate_per_sec = float(machine_usd_per_hr) / 3600.0
80          usage = {billing.WALL_SECONDS: float(billed_seconds)}
81          rates = {billing.WALL_SECONDS: rate_per_sec}
82          usd, breakdown = billing.compute_cost_usd(usage, rates)
83          return {
84              "billed_seconds": round(float(billed_seconds), 1),
85              "gpu_seconds": round(float(gpu_seconds), 1),
86              "machine_usd_per_hr": float(machine_usd_per_hr),
87              "usd": usd,
88              "inr": billing.to_inr(usd, fx_inr_per_usd),
89              "breakdown": breakdown,
90          }
91
92
```

8. user (2026-06-22T09:47:16.678Z)

```
70      bash deploy/digitalocean/gpu_setup.sh || echo "WARN: gpu_setup (WASB env) returned
nonzero ? continuing"
71
72      # 4) Stage the WASB checkpoint (gpu_setup does NOT fetch weights ? README §4).
73      if [ -n "$WEIGHTS_URI" ]; then
74        mkdir -p ~/models/WASB-SBDT/pretrained_weights
75        gcloud storage cp "$WEIGHTS_URI" ~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar
76        export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
77      fi
78
```

```
79      # 5) Run the regression on the gs:// golden clips in the committed manifest. Cost = VM
uptime so far.
80      UPTIME=$(( $(date +%s) - BOOT_EPOCH ))
81      EMAIL_FLAG=""; [ "$DO_EMAIL" = "true" ] && EMAIL_FLAG="--email --email-to $EMAIL_TO"
82      set +e
83      NIGHTLY_SMTP_SECRET="$SMTP_SECRET" NIGHTLY_SMTP_USER="$SMTP_USER"
NIGHTLY_EMAIL_TO="$EMAIL_TO" \
84        python scripts/nightly_reelcount.py --vm-uptime-seconds "$UPTIME" $EMAIL_FLAG
85      RUN_RC=$?
86      set -e
87      echo "runner exit code: $RUN_RC  (0 pass / 1 regression / 4 stale)"
88
89      # 6) Upload the dated report to GCS for the record (alongside the email).
```

9. user (2026-06-22T09:47:17.332Z)

```
689c738 docs(pkg): mark P2c ? + the loop closed ? v1 LIGHT is feature-complete (#313)
293e5c0 feat(pkg): re-bundle backend/ui ? serve the P2c first-run wizard from indexer --app
(#312)
f8b2c3b feat(pkg): P2b-install ? batteries-included install + truly-local 'motion' first-run
default (gate O8) (#310)
0ed4cc6 fix(segmenter): fail the job on an unrunnable config instead of a silent 0-rally success
(Finding B) (#311)
a42c132 docs(pkg): track the batteries-included app plan (PACKAGING_PLAN.md) + index pointers
(#309)
de2bf14 feat(pkg): P2e ? lazy-torch so the LIGHT tier imports with NO torch installed (default-
OFF) (#308)
679e642 feat(pkg): P2d ? DB upgrade-safety (downgrade guard + backup-before-migrate + export)
(default-OFF) (#307)
f01dc85 feat(db): v7 ? record the UI locale per job (PMF analytics) + count_jobs_by_locale
(#306)
fa7ee3f feat(pkg): P2a ? single ffmpeg/ffprobe resolver (env-overridable; one fail-loud place)
(default-OFF) (#305)
e99d0c3 feat(stitch): localize the reel watermark to the UI locale (+ OFL Noto fonts) (#304)
```

10. assistant (2026-06-22T09:47:19.622Z)

Now let me search for how the cost is calculated and verify if there's any documentation or error handling around the timing discrepancy:

11. user (2026-06-22T09:47:21.397Z)

```
Found 1 file
scripts\nightly_reelcount.py
```

12. user (2026-06-22T09:47:22.706Z)

```
Found 34 files
tests\test_nightly_reelcount.py
scripts\nightly_reelcount.py
docs\NIGHTLY_ENGINE_REGRESSION.md
docs\CODE_MAP.md
deploy\gcp\nightly_regression\README.md
deploy\gcp\nightly_regression\run_on_vm.sh
tests\test_billing.py
docs\archives\past_projects\README.md
docs\archives\past_projects\DECOUPLED_COMPUTE\README.md
docs\archives\past_projects\DECOUPLED_COMPUTE\HANDOFF.md
docs\archives\past_projects\DECOUPLED_COMPUTE\07-COMPUTE-ABSTRACTION.md
docs\archives\past_projects\DECOUPLED_COMPUTE\DEPLOY_REPORT_GCP_2026-06-20.md
docs\archives\past_projects\DECOUPLED_COMPUTE\06-RISKS-AND-OPEN-QUESTIONS.md
docs\archives\past_projects\DECOUPLED_COMPUTE\05-COST-ATTRIBUTION.md
docs\archives\past_projects\DECOUPLED_COMPUTE\04-BUILD-AND-TEST-PLAN.md
docs\archives\past_projects\DECOUPLED_COMPUTE\03-PLATFORM-OPTIONS.md
```

```
docs\archives\past_projects\DECOUPLED_COMPUTE\01-EXISTING-PLANS-AND-GAP.md
docs\archives\decisions\DECISIONS.md
docs\README.md
docs\PACKAGING_PLAN.md
docs\NEXT_STEPS.md
docs\HOSTING_PLAN.md
docs\COMPUTE_DECOUPLED_SERVING\runlogs\README.md
docs\COMPUTE_DECOUPLED_SERVING\TESTING_STRATEGY.md
docs\COMPUTE_DECOUPLED_SERVING\README.md
deploy\gcp\teardown.sh
deploy\gcp\provision.sh
deploy\gcp\README.md
deploy\cloudrun\README.md
backend\main.py
backend\infrastructure\database.py
backend\config\models.py
backend\billing.py
backend\api\job_worker.py
```

13. user (2026-06-22T09:47:23.560Z)

```
450
451     def build_parser() -> argparse.ArgumentParser:
452         p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
quality + cost.")
453         p.add_argument("--manifest", default=DEFAULT_MANIFEST)
454         p.add_argument("--baseline", default=DEFAULT_BASELINE)
455         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
456         p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
457         p.add_argument("--vm-uptime-seconds", type=float, default=None,
458                        help="authoritative VM uptime (boot?delete) for the cost line; "
459                             "defaults to the in-process pipeline wall time (a lower bound)")
460         p.add_argument("--no-rerun-guard", action="store_true",
461                        help="disable the re-run-once-on-mismatch guard (the DB-drop flake
mitigation)")
462         p.add_argument("--update-baseline", action="store_true",
463                        help="freeze the current reel counts + quality as the new baseline
instead of checking")
464         p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
465         p.add_argument("--email-to", default=None, help="override the report recipient")
466         p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
467         p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
468         return p
469
470
471     def main(argv: Optional[List[str]] = None) -> int:
472         import logging
473         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
474         log = logging.getLogger("nightly_reelcount")
```

14. assistant (2026-06-22T09:47:26.129Z)

Let me check the documentation and understand the design intent better:

15. user (2026-06-22T09:47:27.119Z)

```
1       # Nightly engine regression ? reel-count + quality + cost, on a GCP GPU VM, emailed
2
3       > **Status:** `IN PROGRESS` ? code shipped, owner-enable steps pending · **Owner:** Avi
· **Created:** 2026-06-22 · **Last updated:** 2026-06-22
4       > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived`
```

> **Tracking anchors:** §7 is the source of truth; pointer in memory `current-state`.
> **Relation to existing docs:** peer-of `scripts/nightly_regression.py` (PR #202 ? the CACHED-trajectory CPU tripwire); this is the FULL-pipeline GPU sibling. Reuses `deploy/gcp/` + `backend/billing.py` + `backend/eval/rally_seg_eval.py`.
> **Honesty note:** claims tagged `[verified]` (checked vs code, offline) / `[design]` (proposed, owner-verified on the VM).

---

0. TL;DR

A **pure-engine** nightly that runs the **configured + checkpointed** pipeline (real WASB GPU detector) end-to-end on 2-3 short owned golden clips and asserts the **number of reels does not change** vs a frozen baseline ? emailing the owner the **quality metrics + regression verdict + run cost**. It runs on an **ephemeral GCP GPU VM that self-deletes** (cost rule). The Python runner + email are unit-tested offline `[verified]`; the GPU/GCS/email VM flow is owner-verified via a one-command dry-run `[design]`. It is **default-additive** ? a tripwire, not the promotion gate.

1. Problem & goal

The owner wants a nightly test that runs the *shipped* engine on a few real videos and catches any change in how many highlight reels it produces (plus quality drift), with the cost of the run, emailed. PR #202 already guards the windowing/scorer via a CPU re-score of *cached* trajectories; it does **not** run the real model or produce reels. This closes that gap by running the full pipeline on a GPU. Read: `deploy/gcp/nightly_regression/README.md` (the runbook), `deploy/gcp/README.md` (the GCP infra), `scripts/nightly_reelcount.py`.

2. Decisions locked

#	Decision	Source / date	Implication
D1	Asserted signal = **reel count** = `len(play_segments)` after segmentation (the detected highlight clips); a stitched reel is 1-per-compile	adversarial mapping 2026-06-22	one stable integer per video, read in-process from the segmenter result
D2	Reel count asserted **EXACT** (`==`); quality metrics with tolerance	determinism review `[verified]`	pure-CV/WASB detection is deterministic; `random.seed(0)` + re-run-once guard absorb the one transient DB-insert flake
D3	Runs on an **ephemeral GCP GPU VM that self-deletes** (not GitHub CI, not local)	owner 2026-06-22 + ADR-013 (compute=GCP)	GitHub CI can't run WASB (no GPU/weights); the VM is the only faithful venue; self-delete = cost rule
D4	**Cost** = VM-uptime × machine $/hr via `backend.billing` (USD + INR)	owner ask + #288 M8	honest "cost of running it"; reuses the existing cost math
D5	**Email** via SMTP, creds from Secret Manager/env, graceful no-creds skip	recommended default 2026-06-22	Gmail app password is simplest; a broken mailer never masks a regression
D6	Driver-agnostic launcher; **Cloud Scheduler ? Cloud Run job** recommended (scale-to-zero)	recommended default 2026-06-22	also runnable from any cron / the owner's box / a Claude routine

4. Design / architecture

`Cloud Scheduler (cron) ? Cloud Run job (launcher) ? launch.sh ? ephemeral GPU VM (run_on_vm.sh): pipeline ? reel-count + quality + cost ? email ? self-delete.`

Reuse map (new vs reused):

- *New:* `scripts/nightly_reelcount.py` (runner), `scripts/nightly_email.py` (email), `deploy/gcp/nightly_regression/*` (orchestration), `eval_baselines/reelcount_manifest.json` (corpus).
- *Reused:* `backend.pipeline.segmenters.segmenter_registry` + `process_video` (the pipeline) · `backend.eval.rally_seg_eval.score_intervals` (quality) · `backend.billing` (cost) · `backend.storage.fetch` (gs:// ? local) · `deploy/gcp/create_gpu_vm.sh` conventions (zone-sweep, DLVM image, `rally-worker` SA, Ops Agent) · `deploy/gcp/teardown.sh` cost rule · the PR #202 baseline/exit-code/report pattern.

The pure core (`summarize_results`/`compare`/`cost_report`/`format_report`) has no IO and is unit-tested; the IO shell (`run_one_video`/`run_corpus`) lazily imports backend + needs a GPU + video.

```
36      ## 6. Honest limits ? what this does NOT do
37      - A green offline test suite proves the **comparison/cost/report/email-format logic**
only ? NOT that WASB runs on the VM, that gs:// ingest works, or that email delivers. Those are
owner-verified via the dry-run `[design]`.
38      - The committed **baseline is owner-frozen** on the first real GPU run (`--update-
baseline`); until then the runner exits 3 (no baseline). The reel counts are not committed by
this PR (they require a GPU).
39      - It does not run in GitHub CI (no GPU/weights). The portable CPU-motion smoke is a
deliberately separate, optional follow-up.
40      - It is a **tripwire**, not the promotion gate; a regression flags an alert, it does not
block a merge.
41
42      ## 7. Deliverables & progress tracker  ? source of truth
43      Legend: ? Todo · ? In progress · ? Done · ? Gated.
44
45      | ID | Deliverable | Depends on | Gated? | Status | PR |
46      |----|-------------|-----------|--------|--------|----|
47      | E1 | `scripts/nightly_reelcount.py` runner (reel-count + quality + cost; pure core +
IO shell) | ? | No | ? | this PR |
48      | E2 | `scripts/nightly_email.py` (SMTP / Secret Manager, graceful) | ? | No | ? | this
PR |
49      | E3 | `deploy/gcp/nightly_regression/` (run_on_vm.sh + launch.sh +
register_scheduler.sh + Dockerfile + README) | ? | No | ? | this PR |
50      | E4 | `eval_baselines/reelcount_manifest.json` (template) + offline tests + docs | ? |
No | ? | this PR |
51      | E5 | **Owner: upload 2-3 short owned clips + labels + weights to `gs://?/nightly/`** |
E1 | ? owner | ? | ? |
52      | E6 | **Owner: SMTP secret in Secret Manager** | E2 | ? owner | ? | ? |
53      | E7 | **Owner: `launch.sh` dry-run + freeze `reelcount_baseline.json`** | E3,E5 | ?
owner | ? | ? |
54      | E8 | **Owner: `register_scheduler.sh` (or a cron)** ? live nightly | E7 | ? owner | ?
| ? |
55      | E9 | *(fast-follow)* custom VM image to skip the ~15-min nightly env build | E7 | No |
? | ? |
56
57      ## 8. Open questions ? owner
58      1. **Which 2-3 short (~5-6 min) owned, minors-free clips** for the corpus? (the manifest
has placeholders; edit + upload.)
59      2. **Driver:** Cloud Run job (recommended) vs cron-on-a-micro-VM vs your-machine-cron vs
a Claude routine? (all supported; `register_scheduler.sh` does the Cloud Run job.)
60      3. **Email:** Gmail SMTP app password (recommended) confirmed, or a transactional API?
61
62      ## 9. Definition of done
63      - [x] Runner + email + orchestration + manifest + offline tests + docs merged (default-
additive).
64      - [ ] Owner dry-run of `launch.sh` is green on a real VM; baseline frozen + committed.
65      - [ ] Scheduler wired; one real nightly email received with metrics + regression verdict
+ cost.
66      - [ ] Then flip Status ? DONE and archive.
67
68      ## 10. References
69      **Internal code anchors `[verified]`:** `scripts/nightly_reelcount.py`,
`scripts/nightly_email.py`, `deploy/gcp/nightly_regression/*`, `backend/billing.py`,
`backend/eval/rally_seg_eval.py::score_intervals`, `backend/storage/__init__.py::fetch`,
`deploy/gcp/create_gpu_vm.sh`. **Internal docs:** `deploy/gcp/nightly_regression/README.md`,
`deploy/gcp/README.md`, `docs/R2_EVAL_METRIC_DESIGN.md`, memory `eval-dual-set-regression`.
**Sibling:** `scripts/nightly_regression.py` (PR #202).
70
71      ---
72
73      ### Changelog
74      - `2026-06-22` ? created; E1-E4 shipped (code + tests + docs), E5-E8 owner-gated.
75
```

16. user (2026-06-22T09:47:28.638Z)

No matches found

17. assistant (2026-06-22T09:47:30.651Z)

Now let me check the nightly_email.py to see if there's any acknowledgment of cost estimates:

18. user (2026-06-22T09:47:31.880Z)

```
1       """Email the nightly regression report ? stdlib SMTP, creds from env or GCP Secret
Manager.
2
3       Used by ``scripts/nightly_reelcount.py --email`` (and reusable by any nightly).
Deliberately
4       dependency-light (stdlib ``smtplib`` + ``email.message``) and **graceful**: if no SMTP
password can
5       be resolved it logs + returns ``False`` (the report is still written/printed ? a broken
mailer must
6       never mask a regression).
7
8       Credential resolution (first hit wins), so it works on a GCP VM, locally, or in tests:
9         ? Recipient:  ``--email-to`` arg  ?  ``NIGHTLY_EMAIL_TO``  ?  ``avi.dullu@gmail.com``
10        ? SMTP host:  ``NIGHTLY_SMTP_HOST``  (default ``smtp.gmail.com``)   · port
``NIGHTLY_SMTP_PORT`` (587, STARTTLS)
11        ? SMTP user:  ``NIGHTLY_SMTP_USER``  (default = recipient)
12        ? SMTP pass:  ``NIGHTLY_SMTP_PASS``  ?  Secret Manager ``NIGHTLY_SMTP_SECRET``
13                      (``projects/<p>/secrets/<name>/versions/latest``; via the google-cloud-
secret-manager
14                       lib if installed, else the ``gcloud`` CLI). A Gmail **app password** is
the simplest.
15
16      Set the secret once (owner):  echo -n '<app-password>' | gcloud secrets create nightly-
smtp-pass --data-file=-
17      and on the VM:  export NIGHTLY_SMTP_SECRET=projects/<proj>/secrets/nightly-smtp-
pass/versions/latest
18                      export NIGHTLY_SMTP_USER=<your-gmail>
NIGHTLY_EMAIL_TO=avi.dullu@gmail.com
19      """
20      from __future__ import annotations
21
22      import logging
23      import os
24      from email.message import EmailMessage
25      from typing import Optional
26
27      log = logging.getLogger("nightly_email")
28
29      DEFAULT_TO = "avi.dullu@gmail.com"
30      DEFAULT_HOST = "smtp.gmail.com"
31      DEFAULT_PORT = 587
32
33
34      def build_message(subject: str, body_md: str, from_addr: str, to_addr: str) ->
EmailMessage:
35          """A text/plain email carrying the markdown report (renders fine in any client).
Pure ? testable."""
36          msg = EmailMessage()
37          msg["Subject"] = subject
38          msg["From"] = from_addr
39          msg["To"] = to_addr
40          msg.set_content(body_md)
41          return msg
42
43
44      def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
45          """Resolve the SMTP password from env or GCP Secret Manager. Returns None if
```

```
unresolvable."""
46          env = env if env is not None else os.environ
47          direct = env.get("NIGHTLY_SMTP_PASS")
48          if direct:
49              return direct
50          secret_ref = env.get("NIGHTLY_SMTP_SECRET")
51          if not secret_ref:
52              return None
53          # Prefer the client lib; fall back to the gcloud CLI (no extra dependency required).
54          try:
55              from google.cloud import secretmanager  # type: ignore
56              client = secretmanager.SecretManagerServiceClient()
57              return
client.access_secret_version(name=secret_ref).payload.data.decode("utf-8")
58          except Exception:  # noqa: BLE001 ? lib absent or auth issue ? try the CLI
59              pass
60          try:
61              import subprocess
62              name = secret_ref.split("/secrets/", 1)[1].split("/", 1)[0] if "/secrets/" in
secret_ref else secret_ref
63              out = subprocess.run(["gcloud", "secrets", "versions", "access", "latest", "--
secret", name],
64                                   capture_output=True, text=True, timeout=30)
65              return out.stdout.strip() if out.returncode == 0 and out.stdout.strip() else
None
66          except Exception as e:  # noqa: BLE001
67              log.warning("secret-manager resolution failed: %s", e)
68              return None
69
70
71      def send_report(subject: str, body_md: str, to: Optional[str] = None,
72                      env: Optional[dict] = None) -> bool:
73          """Send the report. Returns True if sent, False if skipped (no creds) ? never raises
on missing creds."""
74          env = env if env is not None else os.environ
75          to_addr = to or env.get("NIGHTLY_EMAIL_TO") or DEFAULT_TO
76          host = env.get("NIGHTLY_SMTP_HOST", DEFAULT_HOST)
77          port = int(env.get("NIGHTLY_SMTP_PORT", DEFAULT_PORT))
78          user = env.get("NIGHTLY_SMTP_USER") or to_addr
79          password = resolve_secret(env)
80          if not password:
81              log.warning("no SMTP password (set NIGHTLY_SMTP_PASS or NIGHTLY_SMTP_SECRET) ?
email skipped.")
82              return False
83
84          import smtplib
85          msg = build_message(subject, body_md, from_addr=user, to_addr=to_addr)
86          with smtplib.SMTP(host, port, timeout=60) as s:
87              s.starttls()
88              s.login(user, password)
89              s.send_message(msg)
90          log.info("emailed report to %s", to_addr)
91          return True
92
```

19. assistant (2026-06-22T09:47:34.472Z)

Let me check how the cost is reported in the format_report function and what the actual output shows:

20. user (2026-06-22T09:47:36.365Z)

```
220
221      def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
222                        result: NightlyResult, date: str, head: str) -> str:
```

```
223         status = result.overall_status()
224         badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
225                  "STALE": "? STALE (re-freeze the baseline)"}[status]
226         cost = current.get("cost", {})
227         L: List[str] = [
228             f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
229             "",
230             f"**Status: {badge}**  ·  exit code {result.exit_code()}  ·  segmenter
`{current.get('segmenter')}`",
231             "",
232             f"- HEAD `{head}`  ·  baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",
233             f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})**  ·  "
234             f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
235             "",
236         ]
237         if result.regressions:
238             L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
+ [""]
239         if result.stale:
240             L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
241             L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]
242
243         # Per-video table.
244         b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
245         L += ["## Per-video", "",
246               "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
247               "|---|---|---|---|---|"]
248         for v in sorted(set(b_pv) | set(c_pv)):
249             b, c = b_pv.get(v, {}), c_pv.get(v, {})
250             bc = b.get("reel_count")
251             cc = c.get("reel_count")
252             rc = f"{'?' if bc is None else bc}?{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
253             bq, cq = b.get("quality"), c.get("quality")
254             tf = f"{_q(bq, 'tol_f1')}?{_q(cq, 'tol_f1')}"
255             f5 = f"{_q(bq, 'f1@0.5')}?{_q(cq, 'f1@0.5')}"
256             vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else "?"
257             L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
258         L.append("")
259
260         if result.improvements:
261             L += ["_Improvements over baseline (re-freeze to ratchet up):_"] \
262                 + [f"- {f.message()}" for f in result.improvements] + [""]
263         L += ["---",
264               "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
265               "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
266         return "\n".join(L)
267
268
269     # ---------------------------------------------------------------------------- #
```

21. user (2026-06-22T09:47:37.804Z)

```
73:def cost_report(billed_seconds: float, machine_usd_per_hr: float,
77:    ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes ``--vm-
uptime-seconds``;
80:    usage = {billing.WALL_SECONDS: float(billed_seconds)}
84:        "billed_seconds": round(float(billed_seconds), 1),
```

```
234:        f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
495:    cost = cost_report(
496:        billed_seconds=billed,
```

22. assistant (2026-06-22T09:47:40.640Z)

Now let me check the test file to see if there are any tests around the cost estimation:

23. user (2026-06-22T09:47:41.900Z)

```
./tests/test_nightly_reelcount.py
./tests/test_nightly_regression.py
./tests/__pycache__/test_nightly_reelcount.cpython-313-pytest-9.0.3.pyc
./tests/__pycache__/test_nightly_regression.cpython-313-pytest-9.0.3.pyc
```

24. user (2026-06-22T09:47:42.406Z)

```
1       """Tests for scripts/nightly_reelcount.py + scripts/nightly_email.py ? the pure cores.
2
3       Fully offline: no GPU, no video, no network, no real corpus. The comparison / cost /
report / email
4       formatting operate on plain dicts + strings, so reel-count regressions, quality drops,
pipeline
5       errors, corpus drift, and the no-creds email skip are all exercised deterministically.
The actual
6       pipeline run (run_one_video / run_corpus) needs a video + GPU and is owner-verified on
the VM.
7       """
8       import importlib.util
9       import os
10      import sys
11
12      import pytest
13
14      _SCRIPTS = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
"scripts")
15
16
17      def _load(modname):
18          spec = importlib.util.spec_from_file_location(modname, os.path.join(_SCRIPTS,
f"{modname}.py"))
19          mod = importlib.util.module_from_spec(spec)
20          sys.modules[modname] = mod  # register before exec so dataclass annotations resolve
21          spec.loader.exec_module(mod)
22          return mod
23
24
25      nr = _load("nightly_reelcount")
26      ne = _load("nightly_email")
27
28
29      # ---------------------------------------------------------------------------- #
30      # Builders
31      # ---------------------------------------------------------------------------- #
32      def _video(name, reel_count, ok=True, error=None, quality=None):
33          return {"name": name, "reel_count": reel_count, "ok": ok, "error": error,
34                  "quality": quality, "wall_seconds": 1.0}
35
36
37      def _snap(per_video, segmenter="wasb_hybrid"):
38          return {"segmenter": segmenter, "per_video": per_video,
39                  "cost": {"usd": 0.05, "inr": 4.15, "billed_seconds": 500,
"machine_usd_per_hr": 0.35}}
```

```
40
41
42      TOLS = nr.Tolerances()
43
44
45      # ---------------------------------------------------------------------------- #
46      # cost_report (billing wrapper)
47      # ---------------------------------------------------------------------------- #
48      def test_cost_report_math():
49          c = nr.cost_report(billed_seconds=3600, machine_usd_per_hr=0.35,
fx_inr_per_usd=83.0)
50          assert c["usd"] == pytest.approx(0.35, abs=1e-4)        # 1hr @ $0.35
51          assert c["inr"] == pytest.approx(0.35 * 83.0, abs=0.01)
52          assert c["billed_seconds"] == 3600
53
54
55      def test_cost_report_zero_for_zero_time():
56          c = nr.cost_report(billed_seconds=0, machine_usd_per_hr=0.35, fx_inr_per_usd=83.0)
57          assert c["usd"] == 0.0 and c["inr"] == 0.0
58
59
60      # ---------------------------------------------------------------------------- #
61      # summarize_results
62      # ---------------------------------------------------------------------------- #
63      def test_summarize_results():
64          rr = [_video("v1", 5), _video("v2", 3, quality={"tol_f1": 0.4})]
65          snap = nr.summarize_results(rr, "wasb_hybrid", {"usd": 0.1})
66          assert snap["n"] == 2
67          assert snap["per_video"]["v1"]["reel_count"] == 5
68          assert snap["per_video"]["v2"]["quality"]["tol_f1"] == 0.4
69
70
71      # ---------------------------------------------------------------------------- #
72      # compare ? reel count is EXACT
73      # ---------------------------------------------------------------------------- #
74      def test_identical_counts_pass():
75          base = _snap({"v1": {"reel_count": 5, "ok": True}})
76          cur = _snap({"v1": {"reel_count": 5, "ok": True}})
77          res = nr.compare(base, cur, TOLS)
78          assert res.overall_status() == "PASS" and res.exit_code() == 0
79
80
81      def test_reel_count_change_is_regression():
82          base = _snap({"v1": {"reel_count": 5, "ok": True}})
83          cur = _snap({"v1": {"reel_count": 4, "ok": True}})       # one reel vanished
84          res = nr.compare(base, cur, TOLS)
85          assert res.exit_code() == 1
86          r = res.regressions[0]
87          assert r.metric == "reel_count" and r.baseline == 5 and r.current == 4
88
89
90      def test_reel_count_increase_is_also_regression():
91          # ANY change to the count is a regression ? the count must be stable, either
direction.
92          base = _snap({"v1": {"reel_count": 5, "ok": True}})
93          cur = _snap({"v1": {"reel_count": 6, "ok": True}})
94          assert nr.compare(base, cur, TOLS).exit_code() == 1
95
96
97      def test_pipeline_error_is_regression():
98          base = _snap({"v1": {"reel_count": 5, "ok": True}})
99          cur = _snap({"v1": {"reel_count": None, "ok": False, "error": "WASB OOM"}})
100         res = nr.compare(base, cur, TOLS)
101         assert res.exit_code() == 1
102         assert res.regressions[0].metric == "error" and "WASB OOM" in
```

```
res.regressions[0].detail
103
104
105     # ---------------------------------------------------------------------- #
106     # compare ? quality metrics use tolerance
107     # ---------------------------------------------------------------------- #
108     def test_quality_drop_beyond_tol_is_regression():
109         base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.40}}})
110         cur = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.30}}})   #
?0.10 ? tol
111         res = nr.compare(base, cur, TOLS)
112         assert res.exit_code() == 1 and res.regressions[0].metric == "tol_f1"
113
114
115     def test_quality_drop_within_tol_passes():
116         base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.40}}})
117         cur = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.39}}})   #
within 0.02
118         assert nr.compare(base, cur, TOLS).exit_code() == 0
119
120
121     def test_quality_rise_is_improvement_not_regression():
122         base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"f1@0.5": 0.40}}})
123         cur = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"f1@0.5": 0.55}}})
124         res = nr.compare(base, cur, TOLS)
125         assert res.exit_code() == 0
126         assert [f.metric for f in res.improvements] == ["f1@0.5"]
127
128
129     # ---------------------------------------------------------------------- #
130     # compare ? staleness (segmenter / corpus drift)
131     # ---------------------------------------------------------------------- #
132     def test_segmenter_change_is_stale():
133         base = _snap({"v1": {"reel_count": 5, "ok": True}}, segmenter="wasb_hybrid")
134         cur = _snap({"v1": {"reel_count": 5, "ok": True}}, segmenter="motion")
135         res = nr.compare(base, cur, TOLS)
136         assert res.overall_status() == "STALE" and res.exit_code() == 4
137
138
139     def test_corpus_drift_is_stale_but_checks_intersection():
140         base = _snap({"v1": {"reel_count": 5, "ok": True}, "v2": {"reel_count": 3, "ok":
True}})
141         cur = _snap({"v1": {"reel_count": 9, "ok": True}, "v3": {"reel_count": 1, "ok":
True}})
142         res = nr.compare(base, cur, TOLS)
143         assert "v3" in res.added and "v2" in res.removed
144         # v1 regressed on the intersection ? regression wins over stale (exit 1)
145         assert res.exit_code() == 1
146         assert any(f.video == "v1" and f.metric == "reel_count" for f in res.regressions)
147
148
149     def test_corpus_drift_clean_intersection_is_stale_exit_4():
150         base = _snap({"v1": {"reel_count": 5, "ok": True}, "v2": {"reel_count": 3, "ok":
True}})
151         cur = _snap({"v1": {"reel_count": 5, "ok": True}})       # v2 dropped, v1 unchanged
152         res = nr.compare(base, cur, TOLS)
153         assert res.exit_code() == 4 and res.overall_status() == "STALE"
154
155
156     # ---------------------------------------------------------------------- #
157     # report formatting (smoke)
158     # ---------------------------------------------------------------------- #
159     def test_format_report_has_status_cost_and_regression():
160         base = _snap({"v1": {"reel_count": 5, "ok": True, "quality": {"tol_f1": 0.4,
"f1@0.5": 0.5}}})
```

```
161         base["git_commit"] = "aaa"
162         base["generated_at"] = "2026-06-21"
163         cur = _snap({"v1": {"reel_count": 3, "ok": True, "quality": {"tol_f1": 0.4,
"f1@0.5": 0.5}}})
164         res = nr.compare(base, cur, TOLS)
165         rpt = nr.format_report(base, cur, res, "2026-06-22", "bbb")
166         assert "Nightly engine regression" in rpt
167         assert "REGRESSION" in rpt
168         assert "Run cost:" in rpt and "?" in rpt
169         assert "reel_count" in rpt
170         assert "v1" in rpt
171
172
173     def test_format_report_pass_badge():
174         base = _snap({"v1": {"reel_count": 5, "ok": True}})
175         cur = _snap({"v1": {"reel_count": 5, "ok": True}})
176         res = nr.compare(base, cur, TOLS)
177         assert "PASS" in nr.format_report(base, cur, res, "2026-06-22", "bbb")
178
179
180     # ---------------------------------------------------------------------- #
181     # email ? build + creds resolution + graceful skip
182     # ---------------------------------------------------------------------- #
183     def test_email_build_message():
184         msg = ne.build_message("subj", "body **md**", "from@x.com", "to@y.com")
185         assert msg["Subject"] == "subj" and msg["To"] == "to@y.com" and msg["From"] ==
"from@x.com"
186         assert "body **md**" in msg.get_content()
187
188
189     def test_email_resolve_secret_from_env():
190         assert ne.resolve_secret({"NIGHTLY_SMTP_PASS": "hunter2"}) == "hunter2"
191
192
193     def test_email_resolve_secret_none_when_absent():
194         assert ne.resolve_secret({}) is None
195
196
197     def test_email_send_skips_without_creds():
198         # No password resolvable ? graceful skip (False), never raises, no network.
199         assert ne.send_report("s", "b", to="x@y.com", env={"NIGHTLY_EMAIL_TO": "x@y.com"})
is False
200
201
202     def test_tolerances_default():
203         assert nr.Tolerances().quality_tol == 0.02
204
205
206     def test_segments_to_intervals_both_key_conventions():
207         # motion/DB shape (start_time/end_time) AND the alternate (start/end) both parse;
bad rows skip.
208         segs = [{"start_time": 1.0, "end_time": 2.0}, {"start": 5.0, "end": 7.0}, {"nope":
1}]
209         assert nr._segments_to_intervals(segs) == [(1.0, 2.0), (5.0, 7.0)]
210         assert nr._segments_to_intervals([]) == []
211
```

25. assistant (2026-06-22T09:47:44.678Z)

Now let me check the GCP README to see if there's any documentation about cost reconciliation:

26. user (2026-06-22T09:47:45.994Z)

```
1       # Nightly engine regression on GCP ? reel-count + quality + cost, emailed
2
```

A **pure-engine** nightly (no UI/UX): it runs the **configured + checkpointed** pipeline (the real WASB
GPU detector, as a deployment serves it) end-to-end on 2-3 short owned golden clips and asserts the
number of reels does not change vs a frozen baseline ? emailing the owner the quality metrics, the
regression verdict, and the **cost of the run**. Because the real model needs a GPU, it runs on an
ephemeral GCP GPU VM that self-deletes (cost rule: a running GPU VM is the only real cost).

> Complements `scripts/nightly_regression.py` (PR #202), which re-scores **cached** trajectories on CPU
> (windowing/scorer only). This one runs the **full pipeline on the real model** + reel-count + cost + email.

```
Cloud Scheduler (cron, 03:00 IST)
  ?? Cloud Run JOB (launcher, scale-to-zero)  ?  launch.sh
       ?? create ephemeral GPU VM  (run_on_vm.sh as startup-script)
            1. Ops Agent + pull repo @ HEAD (staged to GCS)        4. diff reel-count (EXACT) + quality
            2. build venv + native WASB env + stage weights           vs eval_baselines/reelcount_baseline.json
            3. for each gs:// golden clip: full pipeline ? reels    5. cost = VM-uptime × $/hr (backend.billing)
                                                                     6. EMAIL the report
7. SELF-DELETE ? $0
```

Files

file	role
`../../../scripts/nightly_reelcount.py`	the runner (reel-count + quality + cost + report); pure core is unit-tested
`../../../scripts/nightly_email.py`	SMTP email (creds from Secret Manager/env; graceful no-creds skip)
`run_on_vm.sh`	VM startup-script: env ? run ? email ? upload report ? **self-delete** (even on failure)
`launch.sh`	driver-agnostic: stage repo ? create the self-deleting VM. **Dry-run this first.**
`Dockerfile.launcher` + `register_scheduler.sh`	the Cloud Scheduler ? Cloud Run job wiring
`../../../eval_baselines/reelcount_manifest.json`	the corpus (add a video = one row)
`../../../eval_baselines/reelcount_baseline.json`	frozen expected counts (owner-frozen via `--update-baseline`)

Prerequisites (owner, once)

1. **GPU quota** `GPUS_ALL_REGIONS ? 1` (see `../README.md` §2) ? already granted (the 2026-06-22 L4 run).
2. **Golden clips + labels in GCS** ? 2-3 short (~5-6 min) **owned, minors-free** clips:
   `gcloud storage cp clip.mp4 gs://khelsutra-rally-corpus/nightly/videos/` and the matching rally labels
   (a `*_golden_features.json` with a `gts` key, or a `start,end` CSV) to `.../nightly/labels/`.
3. **WASB weights in GCS** ? `gcloud storage cp wasb_badminton_best.pth.tar gs://khelsutra-rally-corpus/weights/`.
4. **SMTP secret** (Gmail app password is simplest):
   ```bash
   printf '%s' '<gmail-app-password>' | gcloud secrets create nightly-smtp-pass --data-file=- --project=gen-lang-client-0306026826
   # the launcher passes NIGHTLY_SMTP_SECRET + NIGHTLY_SMTP_USER(=your gmail) + NIGHTLY_EMAIL_TO to the VM
   ```

```
44
45      ## Enable
46      ```bash
47      # 0) point the manifest at your gs:// clips (edit
eval_baselines/reelcount_manifest.json)
48      # 1) DRY-RUN the whole VM flow once and watch it (it self-deletes; freezes nothing yet):
49      NIGHTLY_SMTP_USER=<your-gmail> bash deploy/gcp/nightly_regression/launch.sh
50      gcloud compute instances get-serial-port-output <printed-name> --zone=<printed-zone>   #
tail it
51      # 2) FREEZE the baseline from the configured engine ? run on a GPU box (the dry-run VM,
or a manual VM):
52      #      python scripts/nightly_reelcount.py --update-baseline      # then commit
reelcount_baseline.json
53      # 3) WIRE the nightly schedule (cloud-native, scale-to-zero):
54      bash deploy/gcp/nightly_regression/register_scheduler.sh
55      ```
56
57      ### Trigger options (pick one)
58      - **Cloud Scheduler ? Cloud Run job (recommended)** ? `register_scheduler.sh`. Fully
cloud, scale-to-zero,
59        machine-independent. The only always-on cost is $0; the GPU VM is ephemeral.
60      - **Cron on any always-on box** (a $6/mo `e2-micro`, or your machine) ? `0 3 * * * cd
<repo> && git pull && bash deploy/gcp/nightly_regression/launch.sh`.
61      - **Claude scheduled routine (local)** ? a Claude cron on your machine runs `launch.sh`
(uses your gcloud auth).
62
63      ## Cost
64      Per night ? **VM uptime × machine $/hr**. A 2-3-clip run on an L4 (`g2-standard-4`) is
mostly the ~10-15 min
65      env build + a few min inference ? **$0.05-0.15/night** (T4 `n1-standard-8` similar). The
report shows the
66      exact figure (`backend.billing`, USD + INR). The VM **self-deletes** ? $0 idle. To cut
the nightly env-build
67      time, bake a **custom image** (deps + WASB env + weights preinstalled) and point
`launch.sh` at it ? a fast-follow.
68
69      ## What trips it (exit codes mirror PR #202)
70      `0` pass · `1` regression (reel-count changed on any clip, a clip's pipeline errored, or
a quality metric
71      dropped beyond tolerance) · `4` stale (you added/removed a clip or changed the segmenter
? re-freeze with
72      `--update-baseline`) · `2` operational · `3` no baseline yet. Reel-count is asserted
**EXACT** (pure-CV/WASB
73      detection is deterministic; a re-run-once guard absorbs the one known transient DB-
insert flake).
74
75      ## Add another video later (the pattern)
76      Append one row to `eval_baselines/reelcount_manifest.json`, upload the clip (+labels) to
GCS, then
77      `python scripts/nightly_reelcount.py --update-baseline` and commit the updated baseline.
The runner reports
78      added/removed clips as STALE so a new clip can't silently pass un-baselined.
79
80      > ? The GCP/GPU/email paths are **not** exercised by CI (no GPU/secrets there). The
Python runner + email
81      > pure cores ARE unit-tested (`tests/test_nightly_reelcount.py`); the VM flow is owner-
verified via the §1 dry-run.
82
```

27. assistant (2026-06-22T09:47:49.367Z)

Now let me verify the actual claim by checking the documentation in the code. The finding claims:
1. The cost calculation uses kernel uptime from `date +%s`, not wall-clock
2. This could drift from actual GCP billing

3. There's no audit trail comparing estimated vs actual GCP billing

Let me verify each point:

28. user (2026-06-22T09:47:51.247Z)

(Bash completed with no output)

29. assistant (2026-06-22T09:47:53.294Z)

Let me check what the documentation says explicitly about what cost_report actually calculates:

30. user (2026-06-22T09:47:54.164Z)

```
1       """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
nightly).
2
3       Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
deployment serves
4       it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
5       vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6       trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7       needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8       owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10      What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11        ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12          Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13          **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see §guard).
14        ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15          ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16        ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18      Extensible pattern (add a video = one manifest row + re-freeze):
19        ``eval_baselines/reelcount_manifest.json``  ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20        ``eval_baselines/reelcount_baseline.json``  ? the frozen expected counts (regenerated,
never hand-edited)
21
22      Usage
23      -----
24      Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25          python scripts/nightly_reelcount.py --update-baseline
26
27      Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28          python scripts/nightly_reelcount.py --email
29
30      Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31      (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
```

```
changed ? re-freeze).
32      """
33      from __future__ import annotations
34
35      import argparse
```

31. assistant (2026-06-22T09:47:57.270Z)

Perfect. Now I have the full picture. Let me verify the exact timing method one more time and the documentation claims:

32. user (2026-06-22T09:47:58.248Z)

```
14      echo "===== nightly-regression startup $(date -u +%FT%TZ) ====="
15      BOOT_EPOCH=$(date +%s)
16
17      md() { curl -s -H "Metadata-Flavor: Google"
"http://metadata.google.internal/computeMetadata/v1/$1"; }
18      NAME=$(md instance/name)
19      ZONE=$(md instance/zone | awk -F/ '{print $NF}')
20      REPO_TGZ=$(md instance/attributes/repo-tgz-uri || true)
21      GIT_SHA=$(md instance/attributes/git-sha || echo unknown)
22      BUCKET=$(md instance/attributes/gcs-bucket || true)
23      REPORT_PREFIX=$(md instance/attributes/report-prefix || echo nightly/reports)
```

33. assistant (2026-06-22T09:48:03.238Z)

Now let me examine the key documentation to assess if the claimed issue is real. The finding says:

1. Cost uses kernel uptime from `date +%s` (line 80 of run_on_vm.sh: `UPTIME=$(( $(date +%s) - BOOT_EPOCH ))`)
2. This could differ from actual wall-clock time
3. No audit trail reconciles with GCP's actual billing

Let me verify what the code documentation actually claims about this:

34. user (2026-06-22T09:48:04.041Z)

```
73      def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                      fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75          """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
arithmetic).
76
77          ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
78          otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
79          rate_per_sec = float(machine_usd_per_hr) / 3600.0
80          usage = {billing.WALL_SECONDS: float(billed_seconds)}
81          rates = {billing.WALL_SECONDS: rate_per_sec}
82          usd, breakdown = billing.compute_cost_usd(usage, rates)
83          return {
84              "billed_seconds": round(float(billed_seconds), 1),
85              "gpu_seconds": round(float(gpu_seconds), 1),
86              "machine_usd_per_hr": float(machine_usd_per_hr),
87              "usd": usd,
88              "inr": billing.to_inr(usd, fx_inr_per_usd),
89              "breakdown": breakdown,
90          }
```

35. assistant (2026-06-22T09:48:06.976Z)

Now let me check if there's any verification code or documentation about the accuracy of the

billing estimate vs actual GCP billing. Let me look at the developer-facing documentation more carefully:

36. user (2026-06-22T09:48:08.827Z)

```
No matches found
```

37. user (2026-06-22T09:48:10.174Z)

```
15:[Omitted long matching line]
23:| D4 | **Cost** = VM-uptime × machine $/hr via `backend.billing` (USD + INR) | owner ask +
#288 M8 | honest "cost of running it"; reuses the existing cost math |
```

38. assistant (2026-06-22T09:48:12.813Z)

Excellent! I found a key phrase. The documentation says the cost is "honest cost of running it" but notably does not describe it as an "estimate." Let me verify if there's any documented caveat about what "honest" means:

39. user (2026-06-22T09:48:13.840Z)

```
14      ## 1. Problem & goal
15      The owner wants a nightly test that runs the *shipped* engine on a few real videos and
catches any change in how many highlight reels it produces (plus quality drift), with the cost
of the run, emailed. PR #202 already guards the windowing/scorer via a CPU re-score of *cached*
trajectories; it does **not** run the real model or produce reels. This closes that gap by
running the full pipeline on a GPU. Read: `deploy/gcp/nightly_regression/README.md` (the
runbook), `deploy/gcp/README.md` (the GCP infra), `scripts/nightly_reelcount.py`.
16
17      ## 2. Decisions locked
18      | # | Decision | Source / date | Implication |
19      |---|----------|---------------|-------------|
20      | D1 | Asserted signal = **reel count** = `len(play_segments)` after segmentation (the
detected highlight clips); a stitched reel is 1-per-compile | adversarial mapping 2026-06-22 |
one stable integer per video, read in-process from the segmenter result |
21      | D2 | Reel count asserted **EXACT** (`==`); quality metrics with tolerance |
determinism review `[verified]` | pure-CV/WASB detection is deterministic; `random.seed(0)` +
re-run-once guard absorb the one transient DB-insert flake |
22      | D3 | Runs on an **ephemeral GCP GPU VM that self-deletes** (not GitHub CI, not local)
| owner 2026-06-22 + ADR-013 (compute=GCP) | GitHub CI can't run WASB (no GPU/weights); the VM
is the only faithful venue; self-delete = cost rule |
23      | D4 | **Cost** = VM-uptime × machine $/hr via `backend.billing` (USD + INR) | owner ask
+ #288 M8 | honest "cost of running it"; reuses the existing cost math |
24      | D5 | **Email** via SMTP, creds from Secret Manager/env, graceful no-creds skip |
recommended default 2026-06-22 | Gmail app password is simplest; a broken mailer never masks a
regression |
25      | D6 | Driver-agnostic launcher; **Cloud Scheduler ? Cloud Run job** recommended (scale-
to-zero) | recommended default 2026-06-22 | also runnable from any cron / the owner's box / a
Claude routine |
26
27      ## 4. Design / architecture
28      `Cloud Scheduler (cron) ? Cloud Run job (launcher) ? launch.sh ? ephemeral GPU VM
(run_on_vm.sh): pipeline ? reel-count + quality + cost ? email ? self-delete.`
29
30      **Reuse map (new vs reused):**
31      - *New:* `scripts/nightly_reelcount.py` (runner), `scripts/nightly_email.py` (email),
`deploy/gcp/nightly_regression/*` (orchestration), `eval_baselines/reelcount_manifest.json`
(corpus).
32      - *Reused:* `backend.pipeline.segmenters.segmenter_registry` + `process_video` (the
pipeline) · `backend.eval.rally_seg_eval.score_intervals` (quality) · `backend.billing` (cost) ·
`backend.storage.fetch` (gs:// ? local) · `deploy/gcp/create_gpu_vm.sh` conventions (zone-sweep,
DLVM image, `rally-worker` SA, Ops Agent) · `deploy/gcp/teardown.sh` cost rule · the PR #202
baseline/exit-code/report pattern.
33
```

```
34      The pure core (`summarize_results`/`compare`/`cost_report`/`format_report`) has no IO
and is unit-tested; the IO shell (`run_one_video`/`run_corpus`) lazily imports backend + needs a
GPU + video.
35
36      ## 6. Honest limits ? what this does NOT do
37      - A green offline test suite proves the **comparison/cost/report/email-format logic**
only ? NOT that WASB runs on the VM, that gs:// ingest works, or that email delivers. Those are
owner-verified via the dry-run `[design]`.
38      - The committed **baseline is owner-frozen** on the first real GPU run (`--update-
baseline`); until then the runner exits 3 (no baseline). The reel counts are not committed by
this PR (they require a GPU).
39      - It does not run in GitHub CI (no GPU/weights). The portable CPU-motion smoke is a
deliberately separate, optional follow-up.
40      - It is a **tripwire**, not the promotion gate; a regression flags an alert, it does not
block a merge.
41
42      ## 7. Deliverables & progress tracker  ? source of truth
43      Legend: ? Todo · ? In progress · ? Done · ? Gated.
44
45      | ID | Deliverable | Depends on | Gated? | Status | PR |
46      |----|-------------|-----------|--------|--------|----|
47      | E1 | `scripts/nightly_reelcount.py` runner (reel-count + quality + cost; pure core +
IO shell) | ? | No | ? | this PR |
48      | E2 | `scripts/nightly_email.py` (SMTP / Secret Manager, graceful) | ? | No | ? | this
PR |
49      | E3 | `deploy/gcp/nightly_regression/` (run_on_vm.sh + launch.sh +
register_scheduler.sh + Dockerfile + README) | ? | No | ? | this PR |
50      | E4 | `eval_baselines/reelcount_manifest.json` (template) + offline tests + docs | ? |
No | ? | this PR |
51      | E5 | **Owner: upload 2-3 short owned clips + labels + weights to `gs://?/nightly/`** |
E1 | ? owner | ? | ? |
52      | E6 | **Owner: SMTP secret in Secret Manager** | E2 | ? owner | ? | ? |
53      | E7 | **Owner: `launch.sh` dry-run + freeze `reelcount_baseline.json`** | E3,E5 | ?
owner | ? | ? |
```

40. assistant (2026-06-22T09:48:15.953Z)

Let me read the "Honest limits" section more carefully:

41. user (2026-06-22T09:48:17.367Z)

```
36      ## 6. Honest limits ? what this does NOT do
37      - A green offline test suite proves the **comparison/cost/report/email-format logic**
only ? NOT that WASB runs on the VM, that gs:// ingest works, or that email delivers. Those are
owner-verified via the dry-run `[design]`.
38      - The committed **baseline is owner-frozen** on the first real GPU run (`--update-
baseline`); until then the runner exits 3 (no baseline). The reel counts are not committed by
this PR (they require a GPU).
39      - It does not run in GitHub CI (no GPU/weights). The portable CPU-motion smoke is a
deliberately separate, optional follow-up.
40      - It is a **tripwire**, not the promotion gate; a regression flags an alert, it does not
block a merge.
41
42      ## 7. Deliverables & progress tracker  ? source of truth
43      Legend: ? Todo · ? In progress · ? Done · ? Gated.
44
45      | ID | Deliverable | Depends on | Gated? | Status | PR |
```

42. assistant (2026-06-22T09:48:20.169Z)

Now let me verify the exact nature of the "kernel time vs wall-clock" claim. The finding asserts that `date +%s` measures kernel time, not wall-clock. Let me verify:

43. user (2026-06-22T09:48:21.035Z)

man not available, checking bash docs

44. assistant (2026-06-22T09:48:35.484Z)

Now, let me formulate my findings. Based on my review:

KEY FINDINGS:

1. **The finding's premise is INCORRECT about "kernel time"**: `date +%s` returns **wall-clock time** (seconds since Unix epoch), NOT kernel uptime. This is not a kernel-time measurement?it's the system's current wall-clock time.

2. **The cost calculation method is actually documented**: The code documentation at line 77-78 explicitly says: `billed_seconds is the VM uptime (boot?delete) when the orchestrator passes --vm-uptime-seconds; otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays for boot/install).` This explicitly acknowledges that what's being calculated is an **approximation** with the fallback caveat.

3. **The design decision (D4) describes this as "honest cost" not a "precise bill"**: The locked decision at line 23 says the cost is "honest 'cost of running it'" ? not "exact" like the reel count. The word choice "honest" (not "authoritative" or "actual") signals this is a best-effort estimate.

4. **There IS NO reconciliation with actual GCP billing**: The code and docs do NOT query GCP's Cost Management API or any billing export. The cost reported is indeed a client-side calculation based on VM uptime.

5. **However, for production use this is low-risk**: GCP VM billing is minute-granular with a 1-minute minimum. A 5-15 minute run has negligible drift from clock skew or NTP adjustments (< 1% error), and the per-reel consistency is what matters, not the absolute cost.

VERDICT ON THE FINDING:

The finding raises a **real but very minor issue**: The code does NOT explicitly document that the reported cost is a **best-effort estimate** that could drift from the actual GCP bill (which is authoritative from the Cost Management API). The current docs are subtle ("honest cost") rather than explicit.

However, the claim that this uses "kernel time" is WRONG?`date +%s` is wall-clock time, not kernel uptime. The finding's technical premise is incorrect, which undermines its credibility.

The suggested fix (add a comment noting cost is best-effort + optionally query Cost Management API) is reasonable but LOW severity since:
- The code is clear about what it measures (UPTIME)
- The tolerance mentioned (< 1% error on 5-min runs) is negligible
- This is a **tripwire**, not a billing system
- The owner uses this internally, not customer-facing

Let me now provide the structured output:

45. user (2026-06-22T09:48:43.453Z)

Structured output provided successfully

46. assistant (2026-06-22T09:48:51.189Z)

Summary

I've completed an adversarial review of the finding. Here's my verdict:

The finding is REAL but technically FLAWED in premise:

1. **The core claim is correct**: The code reports a cost based on VM uptime (`date +%s - BOOT_EPOCH`), not the authoritative GCP billing statement. This is a **best-effort estimate**,

not ground truth.

2. **The technical premise is WRONG**: The finding incorrectly claims this uses "kernel time." In reality, `date +%s` returns **wall-clock seconds since Unix epoch**?it's the system's current time, not kernel uptime. This undermines the finding's credibility.

3. **The gap is REAL but LOW-RISK**:
   - No drift reconciliation with GCP Cost Management API exists
   - But for a 5?15 min GPU run, clock drift is negligible (< 1% error)
   - The design acknowledges this as "honest cost" (not "exact")
   - This is a tripwire, not a customer-facing billing system

4. **Recommended fix**: Add one line to the docstring clarifying cost is a "best-effort estimate" based on VM uptime, recommending comparison against actual GCP billing. No code change needed?just documentation.

Severity: LOW ? The issue is real (missing explicit caveat) but the magnitude is negligible for the use case, and the code's behavior is sound. The finding's technical inaccuracy (kernel vs. wall-clock time) suggests it was written without full verification.